



C++ PROGRAMMING

(335)

NATIONAL 2023

APPLICATION KNOWLEDGE:

Inventory and Investment Management _____ (400 points)

TOTAL POINTS _____ (400 points)

Test Time: 90 minutes

Inventory and Investment Management

Many investment opportunities require significant sums of money and resources to fulfill – and those resources will commonly vary in price over time. In this exercise, you will create a C++ console application that manages investments with respect to an available inventory of various resources to determine if the current “market” would support a particular investment.

To make this work, there will be a text file containing item data, and another that will contain a list of investment opportunities. The goal of the application will be to load both files and then report on the cost of acquiring everything needed for an investment, the value of what that investment will produce, and how repeatable such an investment would be.

Provided to the contestant is a sample item database (in reality, this would be sourced by some API that updates hourly or daily, and this application would report on lucrative opportunities). The first value is the recorded value of the item, the second value is the name of that item, and the last value is how many of those items are available for purchase.

```
1000,"Simple Wood",40
90,"Small Stone",74
1780,"Salmon Fillet",172
80,"Salt and Pepper",13
3000,"Baked Salmon",0
145000,"Electric Motor",3
40000,"Control Board",9
275000,"Battery System",2
```

Figure 1: Item data sample information

The investment database provided has the below format, where the first line of a record is the name of the investment itself (not necessarily what it produces). Then, there is quantity information (negative quantity means that it consumes that item, positive quantity means that it produces that item), and the name of the item produced/consumed.

```
"Basic Shed"
-10;"Simple Wood"
-5;"Small Stone"
-15;"Nails"
+1;"Basic Shed"
```

Figure 2: Sample investment information

Once these values are generated, they will be reported in a manner like what is shown below, reporting the name, required value, produced value, profit value, # of times repeatable, and profit %.

INV_NAME	REQ_VAL(\$)	PROD_VAL(\$)	PROFIT(\$)	REP_CNT	PROFIT(%)
Generator	5000.00	9000.00	4000.00	2	1.8000
Basic Shed	106.00	100.00	-6.00	4	0.9434
Baked Salmon	38.00	60.00	22.00	4	1.5789
Personal Computer	5600.00	8000.00	2400.00	1	1.4286
Ice Cream	5.30	28.00	22.70	2	5.2830
Bookshelf	44.00	300.00	256.00	10	6.8182
Practice Carpentry	1703.00	0.00	-1703.00	1	0.0000
Press any key to continue . . .					

Figure 3: Sample output using provided text input

Requirements:

1. You must create a C++ console application named CPP_335_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CPP_335_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. A class for the item data type shall be created for the following:
 - a. To store name and quantity information
 - b. To provide getters and proper constructors for member variables
4. A data type for storing an item data object and the associated value (item value pair) shall also be created, with appropriate constructors/getters.
5. An inventory class shall be created for the purpose of storing a list of item value pairs
 - a. The pairs shall be accessible via string internally (non-integer-based indexing)
 - b. The class shall have a function for loading the inventory from a specified file
6. An investment class shall be created
 - a. Shall store a list of items that are required and another list for items that are produced
 - i. Shall provide suitable add item functions for both needed and produced item lists
 - b. Shall store the name of the investment opportunity, as indicated in the data file, and have suitable constructors/getters
 - c. Shall provide functions for calculating the value based on the currently available inventory and value data
 - i. Shall return required value, produced value, and repeatability count
 1. Repeatability count refers to the number of times that a particular investment can be repeated with the current inventory, based on what items are required
7. A function shall be created for loading the list of investments from a file and constructing a list of investment objects
8. A function shall be created for generating the described report based on the inventory information and list of investment opportunities
9. When finished reporting, the application shall pause and wait for the user to press a key to exit the application.

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive	_____ 10 pts
The project is named according to the naming conventions	_____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB	_____ 10 pts
--	--------------

If the program does not execute, then the remaining items in this section receive a score of zero.

Application displays error message if cannot load file	_____ 10 pts
Application successfully loads list of items/investments	_____ 30 pts
Application generates report in format provided	_____ 20 pts
Proper value calculations are performed using item database	_____ 40 pts
Repeatability count is properly performed	_____ 40 pts
Application pauses once execution is complete	_____ 20 pts

Source Code Review

Code is commented at the top, for each function, and as needed	_____ 20 pts
Code uses reasonable and consistent variable naming conventions	_____ 20 pts
Item class is implemented according to specification	_____ 30 pts
Item Value Pair data type is implemented according to specification	_____ 20 pts
Inventory class is implemented according to specification	_____ 40 pts
Investment class is implemented according to specification	_____ 30 pts
File error handling is performed properly	_____ 30 pts
Application is properly modular in design	_____ 20 pts

Total Points: 400 pts

Completed Source Code

```
#include <iostream>
#include <vector>
#include <map>
#include <fstream>
#include <string>
#include <sstream>
#include <iomanip>

using namespace std;

const string itemDataPath = "itemdata.txt";

// Splits a selected string into a collection
// of strings separated by a given delimiter
vector<string> Split(char delim, string input)
{
    vector<string> tokens; // Collection of string tokens
    stringstream stream(input); // Stringstream of input
    string token; // Used to pull out individual tokens
    // While delims still present in string...
    while (getline(stream, token, delim))
        if (token.compare(""))
            tokens.push_back(token); // Add to vector
    return tokens; // Returns collection
}

class item
{
public:
    item() { name = ""; qty = 0; }
    item(string newName, int newQty)
    {
        name = newName;
        qty = newQty;
    }

    string getName()
    {
        return name;
    }

    int getQty()
    {
        return qty;
    }
private:
    string name;
    int qty;
};

struct itemValPair
{
    item itm;
    int val;

    itemValPair() { itm = item(); val = 0; }
    itemValPair(item newItm, int newVal)
```

```
        {
            itm = newItem;
            val = newVal;
        }
};

class itemList
{
public:
    itemValPair retrievePair(string itemName)
    {
        return items[itemName];
    }

    void loadInventory(string fileName)
    {
        string line;
        ifstream dataFile(fileName);
        while (getline(dataFile, line))
        {
            vector<string> itemTokens = Split(',', line);
            itemTokens[1].erase(remove(itemTokens[1].begin(), itemTokens[1].end(),
            '\\"'), itemTokens[1].end());

            int val = stoi(itemTokens[2]);
            int qty = stoi(itemTokens[0]);

            items.insert(make_pair(itemTokens[1], itemValPair(item(itemTokens[1],
            val), qty)));
        }
    private:
        map<string, itemValPair> items;
    };

    class investment
    {
    public:
        investment(string newName)
        {
            newName.erase(remove(newName.begin(), newName.end(), '\\\"'), newName.end());
            invName = newName;
        }

        void addItemNeeded(string source)
        {
            source.erase(remove(source.begin(), source.end(), '\\\"'), source.end());
            vector<string> itmToken = Split('; ', source);
            item itm(itmToken[1], stoi(itmToken[0]));
            itemsRequired.push_back(itm);
        }

        void addItemProduced(string source)
        {
            source.erase(remove(source.begin(), source.end(), '\\\"'), source.end());
            vector<string> itmToken = Split('; ', source);
            item itm(itmToken[1], stoi(itmToken[0]));
            itemsProduced.push_back(itm);
        }
    };
};
```

```
void getValue(itemList inv, int& requiredVal, int& producedVal, int& repeatCnt)
{
    int repeat = -1;

    for (int i = 0; i < itemsRequired.size(); i++)
    {
        item itm = itemsRequired[i];
        itemValPair pair = inv.retrievePair(itm.getName()); // lookup value of
item in inventory
        requiredVal += pair.val * itm.getQty(); // retrieve necessary quantity

        int countInv = pair.itm.getQty();

        if (countInv >= itm.getQty())
        {
            int newRep = countInv / itm.getQty();

            if (newRep < repeat || repeat == -1)
            {
                repeat = newRep;
            }
        }
        else
        {
            repeat = 0;
        }
    }

    for (int i = 0; i < itemsProduced.size(); i++)
    {
        item itm = itemsProduced[i];
        itemValPair pair = inv.retrievePair(itm.getName()); // lookup value of
item
        producedVal += pair.val * itm.getQty(); // retrieve necessary quantity
    }

    repeatCnt = repeat;
}

string getName()
{
    return invName;
}

private:
    string invName;
    vector<item> itemsRequired;
    vector<item> itemsProduced;
};

void loadOpportunities(vector<investment>& opplList, string filePath)
{
    string line;
    ifstream dataFile(filePath);

    while (getline(dataFile, line))
    {
        if (!line.empty())
```

```
{
    investment inv(line);

    while (getline(dataFile, line))
    {
        if (!line.empty())
        {
            if (line[0] == '-')
            {
                inv.addItemNeeded(line.erase(0, 1));
            }
            else if (line[0] == '+')
            {
                inv.addItemProduced(line.erase(0, 1));
            }
        }
        else
        {
            break;
        }
    }

    oppList.push_back(inv);
}

}

void reportOpportunities(vector<investment> opps, itemList inv)
{
    cout
        << right << setw(25) << "INV_NAME"
        << setw(15) << "REQ_VAL($)"
        << setw(15) << "PROD_VAL($)"
        << setw(15) << "PROFIT($)"
        << setw(15) << "REP_CNT"
        << setprecision(4) << fixed << setw(15) << "PROFIT(%)" << endl;

    for (int i = 0; i < opps.size(); i++)
    {
        investment op = opps[i];

        int requiredVal = 0;
        int producedVal = 0;
        int repeatCnt = 0;
        double profitMargin = 0.0;

        op.getValue(inv, requiredVal, producedVal, repeatCnt);

        profitMargin = (static_cast<double>(producedVal) /
static_cast<double>(requiredVal));

        cout
            << fixed << right << setw(25) << op.getName()
            << setprecision(2) << setw(15) << static_cast<double>(requiredVal) /
100.0
            << setprecision(2) << setw(15) << static_cast<double>(producedVal) /
100.0
            << setprecision(2) << setw(15) << static_cast<double>(producedVal-
requiredVal) / 100.0
```

```
        << setw(15) << repeatCnt  
        << setprecision(4) << setw(15) << profitMargin << endl;  
    }  
}  
  
int main()  
{  
    itemList availableInventory;  
    vector<investment> opportunities;  
  
    availableInventory.loadInventory("itemdata.txt");  
    loadOpportunities(opportunities, "investments.txt");  
  
    reportOpportunities(opportunities, availableInventory);  
  
    system("pause");  
    return 0;  
}
```